COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	10/06/2026	**DATE SUBMITTED:**	10/07/2026
NAME:	Canoy Hail B. Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11.1 **Basic Algorithm Analysis**

. PROCEDURE OUTPUT

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

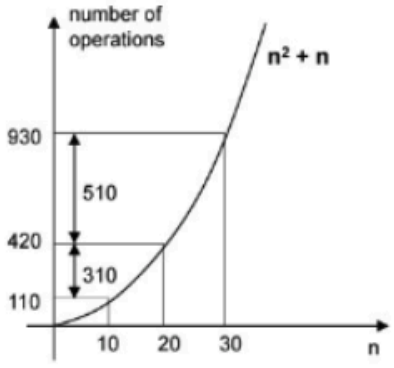
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: _____

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

So basically the number of operations increases as the size of the array increases. The formula is n^2 + n meaning if the size of the array is 10 there will be 10^2 + 10 or 110 operations needed. Then for the others 20^2 + 20 = 420

and 30^2 + 30 = 930 basically it shows how the number of operations increases quickly as the array becomes larger. So in the worst case the algorithm has to check every element in the array x for every element in array meaning its O(n^2).

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

```cpp
#ifndef ILOB_H
#define ILOB_H

#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>


class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;


public:
    //constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v)
    }
    // Getters
    auto get_name() const {
        return name;
    }
    auto is_vaccinated() const {
        return vaccinated;
    }
    // Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }
    // The ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }
    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};
```

```cpp
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);


    // Generate random credentials
    auto random_name = std::make_pair(
        uniform_dist(rand),
        uniform_dist(rand)
    );


    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;


    return Student(random_name, is_vaccinated);
}
```

```cpp
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();


    while (first != last) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = range_length / 2;
        auto mid_element = *(first + mid_element_index);
        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P &&
            mid_element.is_vaccinated() == false) {
            return true;
        }
        else if (mid_element == P &&
                 mid_element.is_vaccinated() == true) {
            return false;
        }
        else if (mid_element > P) {
            last = first + mid_element_index;
        }


        else if (mid_element < P) {
            first = first + mid_element_index + 1;
        }
    }
    // Student not found in the sequence and therefore should be vaccinated
    return true;
}
```

```cpp
void search_test(int size, Student p) {
    std::vector<Student> people;
    // Create a list of random people
    for (auto i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));
    }


    std::sort(people.begin(), people.end());
    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
        std::chrono::steady_clock::now();

    bool search_result = needs_vaccination(p, people);

    std::chrono::steady_clock::time_point end =
        std::chrono::steady_clock::now();
    // Stop the clock
    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>
                 (end - begin).count()
              << " microseconds" << std::endl;


    if (search_result)
        std::cout << "Student ("
                  << p.get_name().first << " "
                  << p.get_name().second
                  << ") needs vaccination."
                  << std::endl;
    else
        std::cout << "Student ("
                  << p.get_name().first << " "
                  << p.get_name().second
                  << ") does not need vaccination."
                  << std::endl;


    std::cout << std::endl;
}


#endif
```

```cpp
#include "ILOB.h"

int main() {
    auto p = generate_random_Student(1000);

    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);

    return 0;
}
```

```
Time taken to search = 0 microseconds
Student (310 115) needs vaccination.

Time taken to search = 1 microseconds
Student (310 115) needs vaccination.

Time taken to search = 1 microseconds
Student (310 115) needs vaccination.


Press any key to continue . . .
```

Observation: The program successfully uses binary search to determine whether a student needs vaccination, and the output shows the search time for different input sizes. As the number of students increases from 1,000 to 10,000 and 100,000, the search remains fast because binary search reduces the search range by half in each step. The changes made in the code include organizing the program into ILOB.h and ILOBmain.cpp, adding the required Student class, random student generation, sorting, vaccination checking, and chrono for measuring runtime. The output shows that the program is efficient and supports the theoretical O(log N) performance of binary search.

Table 11.1 - Code snippet of .cpp/.h file with output of ILOB

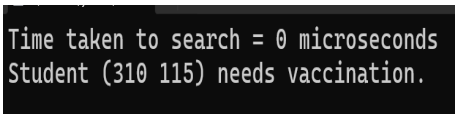
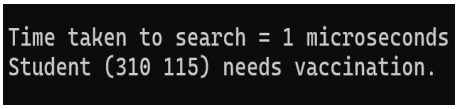
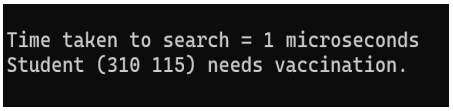
Input Size (N)	Execution Speed	Screenshot	Observation
1000	0 microsecond		The search is very fast because binary search reduces the search range by half each step.
10000	1 microsecond		Even though the input size is larger, the search time remains very small because binary search is efficient.
100000	1 microsecond		The input size is much larger, but the execution time is still the same with previous Input size showing the efficiency of binary search, but has a slight delay because of the larger size.

Table 11.2 - For ILOB

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Measure the runtime of algorithms using theoretical analysis & ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

For each of the problems below, provide the following:

- Theoretical Analysis
- Experimental Analysis
- Analysis and Comparison

Problem 1: Consider a program that returns true if the elements in an array are unique.

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

Theoretical Analysis	The Unique(A) algorithm uses two nested loops to compare every element with the elements that come after it. In the worst case, the algorithm performs approximately n(n−1)/2n(n-1)/2 comparisons. Therefore, its time complexity is $O(n^2)$. The best case is O(1) when a duplicate is found immediately.

Experimental Analysis	When the program is tested with different array sizes, the execution time generally increases as the number of elements increases. As the input size becomes larger, more pairs of elements need to be compared. For example, increasing the input size from 1,000 to 10,000 can cause a significant increase in runtime because of the quadratic number of comparisons.
Analysis and Comparison	The experimental results are consistent with the theoretical analysis because the runtime increases rapidly as the input size increases. The nested loops are responsible for the $O(n^2)$ complexity since each element may need to be compared with many other elements. Therefore, the experimental results demonstrate that the Unique(A) algorithm becomes less efficient for large arrays.

Problem 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Theoretical Analysis	The **rpower** algorithm has a time complexity of O(n) because it decreases n by 1 in every recursive call. The **brpower** algorithm has a time complexity of O(log n) because it reduces n by approximately half in every recursive call.
Experimental Analysis	The runtime of **rpower** increases as the value of n increases because it requires more recursive calls. In comparison, **brpower** remains faster because it requires fewer recursive calls by dividing the problem size by two.
Analysis and Comparison	The experimental results support the theoretical analysis. **rpower** is less efficient for large values of n because it has O(n) complexity, while **brpower** is more efficient with O(log n) complexity. Therefore, **brpower** performs better than **rpower**, especially when the value of n is large.

Conclusion (Canoy Hail B.)

For my conclusion and learnings, I learned that algorithms with lower growth rates are generally more efficient, especially when the input size becomes larger. I also learned that the algorithm for comparing two arrays has an $O(n^2)$ worst-case complexity because it may need to search through one array for every element of the other array.

Conclusion (Mauro, Ryan Lewis B)

In this activity I learned how to analyze the runtime performance of different algorithms using both theoretical and experimental analysis. I learned that the Unique algorithm has $O(n^2)$ complexity because it uses nested loops, while rpower has O(n) complexity because it reduces the exponent by one in each recursive call. I also learned that brpower is more efficient with O(log n) complexity because it reduces the exponent by half through exponentiation by squaring. I understood that algorithms with lower time complexity are generally more efficient, especially when the input size becomes larger.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **none**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **none**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **none**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.